

Panel de Análisis de Rendimiento Web con Svelte + SvelteKit + Gráficos Avanzados.

Proyecto: *“VelocityDash - Velocidad que transforma datos en acción”.*

Objetivo general: Construir un dashboard de rendimiento web con gráficos interactivos avanzados, animaciones fluidas usando Svelte, y datos en tiempo real vía WebSocket.

Introducción – El valor de la velocidad en la web.

Vivimos en un mundo donde cada milisegundo cuenta. Los/as usuarios/as esperan que las páginas web carguen en menos de 2 segundos; si no, abandonan. Pero, ¿cómo medimos y mejoramos el rendimiento real de una aplicación? Ahí entran las “Core Web Vitals” (LCP, FID, CLS) y otras métricas clave.

VelocityDash nace como una herramienta de monitoreo para desarrolladores y equipos de producto. No solo muestra números, sino que los convierte en gráficos interactivos, alertas y acciones. Usaremos Svelte y SvelteKit por su capacidad para crear interfaces ultra reactivas y animaciones nativas sin necesidad de librerías externas pesadas.

¿Qué aprenderán y repasarán?

- Svelte y su sistema de reactividad único (el compilador convierte el código en DOM imperativo).
- SvelteKit para enrutamiento basado en archivos, carga de datos y endpoints.

Proyecto, Diseño e Implementación de Sitios Web Dinámicos.

- Transiciones y animaciones nativas (`fade`, `fly`, `slide`, `crossfade`, animaciones personalizadas).
- Gráficos avanzados con D3.js o Chart.js integrados con Svelte.WebSockets para actualizaciones en tiempo real.
- Store reactivo global (similar a Pinia pero nativo de Svelte).

Este proyecto agreguenlo a su portfolio y, además, forma parte del sistema integrador del segundo cuatrimestre, donde deberán combinar todas las tecnologías vistas en el año. Al final, lo defenderán explicando cada línea de código y su propósito.

Reactividad en Svelte.

Svelte es diferente a React o Vue: no usa un DOM virtual, sino que compila el código en actualizaciones DOM imperativas. La reactividad se basa en:

- **Declaraciones reactivas:** `\$:` (etiquetas reactivas). Se ejecutan cuando sus dependencias cambian.
- **Stores:** Contenedores de estado compartido con suscripción (`writable`, `readable`, `derived`).

Ejemplo básico:

```
<script>
let count = 0;
$: doubled = count * 2;    // Se actualiza automáticamente
$: {
  console.log(`El contador es ${count}`);
}
</script>
```

Proyecto, Diseño e Implementación de Sitios Web Dinámicos.

```
<button on:click={() => count++}>Clickeame</button>
<p>{count} * 2 = {doubled}</p>
```

Transiciones y Animaciones Nativas.

Svelte incluye animaciones listas para usar:

- **`fade`**: Desvanecido.
- **`fly`**: Movimiento desde una posición.
- **`slide`**: Deslizamiento vertical.
- **`crossfade`**: Transición entre elementos con el mismo key.
- **`scale`**: Escalado.

Además, podemos personalizar con **`easing`** (cubic-bezier, elastic, etc.) y **`duration`**, **`delay`**.

Uso en un elemento:

```
<p in:fly={{ y: 20, duration: 500 }} out:fade>Hola</p>
```

Animaciones con **`animate:flip`**: Para reordenar elementos (ej. lista ordenable).

SvelteKit: Routing y Carga de Datos.

SvelteKit es el framework full-stack para Svelte. Sus características clave:

- **File-based routing**: **`src/routes`** → cada archivo **`\*.svelte`** o carpeta crea una ruta.
- **`+page.svelte`**: Página principal de una ruta.
- **`+layout.svelte`**: Layout compartido.

Proyecto, Diseño e Implementación de Sitios Web Dinámicos.

- ``+page.js` / `+page.server.js``: Carga de datos (``load`` function). Puede ser en el servidor o en el cliente.
- **Endpoints**: Archivos ``+server.js`` que devuelven JSON (API).

Ejemplo de ``load`` en ``+page.js``:

```
/* @type {import('./$types').PageLoad} */
export async function load({ fetch }) {
  const res = await fetch('/api/metrics');
  const metrics = await res.json();
  return { props: { metrics } };
}
```

Gráficos Avanzados con D3.js o Chart.js

Podemos integrar librerías de gráficos directamente en Svelte. La clave es:

- Usar ``onMount`` para inicializar el gráfico cuando el elemento esté en el DOM.
- Actualizar el gráfico cuando los datos cambien (usando ``$:`` o suscripciones a stores).

Ejemplo con Chart.js:

```
<script>
import { onMount } from 'svelte';
import Chart from 'chart.js/auto';
export let data;
let canvas;
let chart;
```

```
onMount(() => {  
  chart = new Chart(canvas, {  
    type: 'line',  
    data: data,  
    options: { responsive: true }  
  });  
  return () => chart.destroy();  
});  
  
$: if (chart) {  
  chart.data = data;  
  chart.update();  
}  
</script>  
<canvas bind:this={canvas}></canvas>
```

WebSockets en SvelteKit.

Para datos en tiempo real, podemos abrir una conexión WebSocket en el cliente (o usar librerías como `socket.io`). En SvelteKit, normalmente abrimos el WebSocket en `onMount` y actualizamos un store.

```
// stores/metrics.js  
import { writable } from 'svelte/store';  
export const realTimeData = writable([]);  
  
<script>  
  import { onMount } from 'svelte';  
  import { realTimeData } from '$lib/stores/metrics';  
  let ws;
```

Proyecto, Diseño e Implementación de Sitios Web Dinámicos.

```
onMount(() => {  
  ws = new WebSocket('wss://api.velocitydash.com/ws');  
  ws.onmessage = (e) => {  
    const newPoint = JSON.parse(e.data);  
    realTimeData.update(arr => [...arr.slice(-50), newPoint]);  
  };  
  return () => ws.close();  
});  
</script>
```

Store Reactiva Global.

Svelte tiene su propio sistema de stores: `writable`, `readable`, `derived`. Se usan con suscripción automática en los templates (prefijando con `\$`).

```
// lib/stores/metrics.js  
import { writable, derived } from 'svelte/store';  
  
export const metrics = writable({ lcp: 0, fid: 0, cls: 0 });  
  
export const averageLCP = derived(metrics, $metrics => $metrics.lcp / 1000);
```

En un componente:

```
<script>  
  import { metrics } from '$lib/stores/metrics';  
</script>  
<h1>LCP: {$metrics.lcp}ms</h1>
```

Ejemplo: "Dashboard de Clima en Tiempo Real".

Proyecto, Diseño e Implementación de Sitios Web Dinámicos.

Para que vean cómo se arma un flujo completo con SvelteKit + WebSocket + gráficos, construyamos un mini dashboard de clima.

Paso 1: Estructura de archivos

```
src/  
├─ routes/  
│   ├─ +layout.svelte  
│   └─ +page.svelte  
├─ lib/  
│   ├─ components/  
│   │   └─ WeatherChart.svelte  
│   └─ stores/  
│       └─ weather.js  
└─ server/  
    └─ weather.js (endpoint)
```

Paso 2: Store de clima

```
// src/lib/stores/weather.js  
  
import { writable } from 'svelte/store';  
  
export const weatherData = writable([]);  
export const currentTemp = writable(0);
```

Paso 3: Página principal (`+page.svelte`)

```
<script>  
  
import { onMount } from 'svelte';  
  
import { weatherData, currentTemp } from '$lib/stores/weather';
```

```
import WeatherChart from '$lib/components/WeatherChart.svelte';
import { fade, fly } from 'svelte/transition';

let ws;

onMount(() => {
  ws = new WebSocket('wss://api.weather.com/ws');
  ws.onmessage = (e) => {
    const data = JSON.parse(e.data);
    weatherData.update(arr => [...arr, data.temp]);
    currentTemp.set(data.temp);
  };
  return () => ws.close();
});
</script>

<div class="dashboard" in:fade>
  <h1>Clima en tiempo real</h1>
  <div class="current" in:fly={{ y: 20 }}>
    <span>Temperatura actual: {$currentTemp}°C</span>
  </div>
  <WeatherChart data={$weatherData} />
</div>
```

Paso 4: Componente de gráfico (WeatherChart.svelte)

```
<script>
import { onMount } from 'svelte';
import Chart from 'chart.js/auto';
export let data;
```

Proyecto, Diseño e Implementación de Sitios Web Dinámicos.

```
let canvas;  
let chart;  
  
onMount(() => {  
  chart = new Chart(canvas, {  
    type: 'line',  
    data: {  
      labels: data.map((_, i) => i),  
      datasets: [{ label: 'Temperatura', data, borderColor: 'orange' }]  
    },  
    options: { responsive: true }  
  });  
  return () => chart.destroy();  
});  
  
$: if (chart) {  
  chart.data.datasets[0].data = data;  
  chart.update();  
}  
</script>  
<canvas bind:this={canvas}></canvas>
```

Este mini proyecto demuestra cómo integrar WebSocket, store reactivo y gráficos. En VelocityDash harán lo mismo pero con métricas web y múltiples gráficos.

Proyecto VelocityDash – Descripción y Requisitos.

Descripción General.

Proyecto, Diseño e Implementación de Sitios Web Dinámicos.

VelocityDash es un panel de análisis de rendimiento web que muestra métricas clave (LCP, FID, CLS, TTFB, etc.) en tiempo real. Los datos se actualizan vía WebSocket y se visualizan con gráficos interactivos (líneas, gauges, mapas de calor). Incluye animaciones suaves, alertas configurables y exportación de reportes PDF.

Requisitos Funcionales (ID)

ID	Requerimiento	Descripción
V1	Dashboard de KPIs	Tarjetas con valores actuales de LCP, FID, CLS, TTFB, incluyendo estado (bueno/regular/malo) y tendencia.
V2	Gráficos interactivos	Al menos 4 tipos: gráfico de líneas, gauge (medidor), mapa de calor y gráfico de barras o área.
V3	Datos en tiempo real	Conexión WebSocket que actualiza los datos cada 2-3 segundos (simulación con <code>setInterval</code> o servicio real).
V4	Animaciones Svelte	Uso de <code>fade</code> , <code>fly</code> , <code>slide</code> , <code>crossfade</code> o <code>animate:flip</code> en tarjetas, gráficos y listas.
V5	Store reactivo global	Gestión del estado con stores de Svelte (<code>writable</code> , <code>derived</code>).
V6	Filtros por rango temporal	Selector para ver datos de última hora, 24h, 7d, 30d (cambia la serie mostrada).
V7	Sistema de alertas	Configurar umbrales para cada métrica; si se superan, mostrar notificación visual (con animación).
V8	Exportación a PDF	Botón para generar un reporte con todos los gráficos y KPIs (usar <code>html2canvas</code> + <code>jsPDF</code> o similar).

Ampliaciones adicionales (para destacar).

Proyecto, Diseño e Implementación de Sitios Web Dinámicos.

- **Rutas SvelteKit:** `+page.js` para carga inicial de datos (histórico).
- **Endpoints:** `+server.js` para servir datos estáticos o simulados.
- **Tipado:** Usar TypeScript para mayor robustez (opcional pero recomendado).

Objetivos del Proyecto.

- **Técnico:** Dominar Svelte y SvelteKit, aplicando reactividad, animaciones y stores.
- **Analítico:** Comprender las Core Web Vitals y su importancia.
- **Práctico:** Integrar WebSockets, librerías de gráficos y exportación.
- **Profesional:** Desarrollar una aplicación full-stack con arquitectura limpia y despliegue en Vercel.

Criterios de Evaluación.

Evaluación Continua.

- **Conexión y participación:** Estar en Live Share, interactuar, hacer preguntas.
- Proyecto SvelteKit con estructura básica.
- Store de métricas implementado.
- Al menos un gráfico funcional (líneas).
- Conexión WebSocket simulada (o real).
- **Calidad de código:** Uso de `\$:` cuando corresponda, stores bien definidos, organización de carpetas.

Evaluación Final.

- **Funcionalidad:** Cumplir con al menos 6 de los 8 requisitos (V1 a V8).
- **Animaciones:** Uso correcto de transiciones y animaciones nativas de Svelte.
- **WebSocket:** Los datos en tiempo real se actualizan correctamente.

Proyecto, Diseño e Implementación de Sitios Web Dinámicos.

- **Gráficos:** Interactividad y actualización fluida.
- **Despliegue:** App funcional en Vercel.
- **Informe técnico:** Documento explicando arquitectura, decisiones, problemas y soluciones.
- **Código limpio:** Tipado (si usa TS), buenas prácticas, comentarios relevantes.

Entregables.

1. Enlace de Live Share: Compartir en el comentario privado de la plataforma antes de empezar a codificar.

2. Avance Parcial: Subir a la plataforma un `.zip` del proyecto (excluyendo `node\_modules`) o enlace a repositorio GitHub con el avance.

3. Entrega Final:

- Repositorio público en GitHub con el código completo y `README.md`.
- URL de la aplicación desplegada en Vercel.
- Informe técnico en PDF o Markdown, que incluya:
 - Introducción y objetivos.
 - Explicación de la arquitectura (stores, componentes, rutas).
 - Capturas de pantalla del dashboard funcionando.
 - Dificultades encontradas y soluciones.
 - Conclusiones y aprendizajes.
- Enlace al video (opcional) de una demo de 2 minutos.

Sistema Integrador y Defensa Final.

Recordatorio clave: Este proyecto no es un trabajo aislado. Forma parte del segundo cuatrimestre, donde deberán integrar todas las tecnologías vistas durante el año (HTML/CSS/JS, frameworks, bases de datos, APIs, etc.) en un sistema más amplio, o bien acoplarlo a su proyecto final.

Proyecto, Diseño e Implementación de Sitios Web Dinámicos.

Al final del cuatrimestre, defenderán su proyecto final, demostrando que entienden cada línea de código y cómo interactúan las diferentes capas. Por eso, en VelocityDash, deben codear como corresponde: usar tipado, comentar partes complejas, mantener un historial de commits claro y escribir código legible y modular. La velocidad y la experiencia de usuario/a son el futuro. Con VelocityDash, no solo monitorearán el rendimiento, sino que aprenderán a construir sistemas/aplicaciones web modernas, reactivas y visualmente impactantes. Cada gráfico, cada animación, cada dato en tiempo real los acerca a ser desarrolladores/as full-stack de primer nivel.